Security is not Well-Defined*

Joel Newman, Tommy He

Abstract

In an attempt to show the messiness and difficulty of defining a security in a unifying fashion to be widely applicable yet not bogged down by minute details, we portray many attempts to defining such a foundation. We begin by considering what one should want in a good definition of security and then proceed onto some first attempts using games. Then, we move onto Turing machine ideas, where each agent could be modelled as an idealized computer; however, these frameworks introduce many complexities such as adversaries, efficienty measurements, and distributed systems antics. As a response, we then discuss efforts at abstract security, which defines everything in a mathematically rigorous and set-theoretic fashion, and extensions.

1 Introduction

A good definition of security (in terms of secure protocols) should be:

- *expressive*, allowing expression of arbitrary protocols and security properties.
- *trustworthy*, reliably representing intuitive definitions of security.
- *faithful*, allowing protocols to be represented in a form not too different from their real world implementation.
- *usable*, with security proofs being achievable, and the framework not being so complex that those attempting to use it are likely to make mistakes in doing so.
- *composable*, enabling larger provably-secure protocols to be built from smaller provable-secure parts.

In this way, we can see a security definition as an *abstraction*, in the design sense – it should allow us to break problems down into digestible parts which then can be rebuilt at a higher level, without us having to worry about the details.

Unfortunately, by this standard, there are no good security definitions. In this paper, we analyze a number of security definitions, each of which a candidate for a grand, unifying definition of security, compare them, and consider why it is so difficult to write a security definition that is good.

*Our original paper was supposed to be on verification of secure multi-party computations. In search of literature in this area, we found many scattered results where we could not even find a suitable model to consider. Hence, we decided to veer off a bit and dive into just the difficulty of finding a model to use for security.

2 The Real / Ideal Paradigm

2.1 Game-Based Security Definitions

A first pass at defining security for protocols in general might simply start by defining a "bad outcome" we want to avoid, like an adversary decrypting a message or causing a protocol to incorrectly calculate a value. A proof shows that an adversary cannot succeed, or only can do so with negligible probability.[1]

These definitions and proofs are known in the field as **game-based**, as the scenario is treated as a formal game between an adversary and a challenger. They are often highly practical to prove, with methods for mechanizing and automating proofs described in the literature [BP06], but they cannot form an adequate basis for our grand unifying definition of security.

First of all, game-based definitions cannot be universalized, as the game often depends on the exact form of security being tested. Secondly, and even more importantly, game-based frameworks only are able to ensure their single "bad outcome" endpoint occurs negligibly often but don't have any inherent way to qualify over all possible attacks. It's very possible for, say, a naïve game defined for encryption systems to miss attacks in which known plaintext is used to find an encryption key, or important portions of the plaintext can be recovered from ciphertext. Thirdly, and potentially most importantly, game-based definitions are in general not able to achieve composability, as described above. This alone is enough to make them unsuitable.

2.2 The Paradigm

To fix these problems, a new concept was introduced, known as either the **real / ideal paradigm** or the **simulation-based model of security**. The concept is as follows: rather than attempt to define security in the form of negatives – "this bad thing does not happen" – define it in terms of a single positive. Compare your *real protocol* to an *ideal protocol*[2] that we construct to be secure by-definition, and prove that attacker's powers are in some sense "the same" between these two worlds. This usually takes the form of an *indistinguishability* or *simulation* proof, in which it is shown that any real attacker or adversary can be simulated by a fake "ideal" attacker in the ideal world. Almost immediately out of this concept falls the definition of "**standalone**" real / ideal paradigm definition of security, as detailed in [Lin17].

This definition is simple but unfortunately insufficient: it only guarantees security is preserved under sequential, not concurrent composition, making it essentially useless for a large class of ideal protocols, such as those for secure or authenticated channels.

The most common fix to this problem comes in two steps. First of all, rather than simply refer to two different situations as being indistinguishable, we explicitly model an entity that is our distinguisher, called the *environment machine*, and define that with overwhelming probability, it does not succeed. The second step is to allow communication back and forth between the environment and the adversary at any time. In this way, the environment comes to represent the entirety of the information that may come from outside the system, and we are able to guarantee security under arbitrary composition.

[1] More precisely, the proof usually is that an adversary that succeeds more than x percent of the time could be used to efficiently compute something y percent of the time that we assume is impossible to efficiently compute more than z percent of the time.

[2] Note that ideal protocols in general are not feasible to execute in the real world, usually requiring a trusted party and perfectly secure communication channels.

3 Turing-Machine Models

3.1 Canetti's Universal Composability

First published in the year 2000, Ran Canetti's Universal Composability Framework (often shortened as *UC*) [Can20] is a field-defining work. Canetti was the first to propose the environmental security strategies as described above for guaranteeing security under arbitrary composition.

The Computation Model Each party of a protocol is modeled as a *collection of* machines, rather than as a single machine; this allows the modeling of a subroutine of some machine as a secondary machine that communicates with it, which is useful for modeling composition. These machines are specified as nondeterministic interactive Turing machines (ITMs): Turing machines with random tapes and the ability to communicate with each other. This communication is done through an addressing system, with ITMs addressing each other by their Session Identifier (SID), Party Identifier (PID), and that recipient's full *code*. If no such ITM is present in the system already (and a special *forced-write* flag is specified in the message) then a new ITM is instantiated with this SID, PID, and code, allowing ITMs to spin up new ITMs with almost arbitrary dynamically-generated code.

Types of Communication Each message sent between IITMs is labeled with a *type*: *input*, *subroutine-output*, or *backdoor*. Backdoor messages are reserved for communication with the adversary, usually either to represent information "leaked" to the public or allow the adversary to compromise a party, usually by asking politely for it to corrupt itself. The other two are used to loosely define a subroutine relation: M is a subroutine of N if M sends subroutine-output to N or M accepts input from M. This subroutine relation is loose, allowing for mutual subroutines or a machine that is a subroutine of more than one other machine. However, certain security theorems only apply to protocols that follow stricter rules for subroutine organization, and it is indeed intended that when a machine communicating directly with another machine, one is a subroutine of the other; communication channels between main parties is intended to be represented by an ITM that is a subroutine of both.[3]

The Environment and Adversary For any protocol π, adversary $\mathcal{A}$, and environment $\mathcal{E}$, in the most recent version of the UC paper[4], we run our mental experiment in the following manner: at the beginning, $\mathcal{E}$ starts executing (potentially initialized with some auxiliary input z) and is the only ITM in the system. $\mathcal{E}$ is then allowed to instantiate (and subsequently communicate with) a single instance of $\mathcal{A}$ and as many instances of ITMs running π as it wants, all having the same SID, with this rules enforced using a mechanism known as the *control function*. It also allows $\mathcal{E}$ and $\mathcal{A}$ to disguise their identities while sending messages while forcing each π ITM to reveal its own while doing the same. Eventually, $\mathcal{E}$ outputs a single bit externally (we can consider this $\mathcal{E}$'s decision as to whether it is interacting with the real or ideal world), and halts.

[3] Note that this allows for an adversary to corrupt/"hack" the communication channel without doing the same to the parties using it, a common attack in the real world.

[4] More on this later

Security We define that π **UC-realizes** (read as "securely computes according to UC") a functionality $\mathcal{F}$[5] if for any adversary for any adversary $\mathcal{A}$ there exists another $\mathcal{S}$ such that for all environments $\mathcal{E}$, satisfying certain "balance" properties, the output of the experiment on $\mathcal{E}$, $\mathcal{A}$, and π is statistically indistinguishable from that on $\mathcal{E}$, $\mathcal{S}$, and $\mathcal{F}$, quantified across all possible auxiliary information z, modulo polynomial runtime notions beyond the scope of this paper.

Composability Unfortunately, we don't quite get to universal composability this easily. The composability theorem only applies to certain classes of protocols that are well-behaved with regard to subroutine organization and allow the adversary to learn the subroutine structure of the system at will, and is parameterized across restrictions on identifiers. Once these are met, we finally get theorems allowing us to guarantee that a protocol is secure when composed arbitrarily into other protocols.

3.1.1 Discussion

Canetti's UC framework is detailed, well-elaborated, and highly expressive. The design decisions of the framework, even the idiosyncratic-seeming ones, are often well-motivated: as one particular example, the model being designed to use ITMs to represent communication channels between parties rather than simply having them communicate directly by sending messages through their communication tapes enables communication channels *themselves* to be modeled as functionalities for which UC-secure protocols may be written. The paper shows this feature off by defining a number of communication models (authenticated communication, secure communication, and synchronous communication) *as* UC-framework functionalities ($\mathcal{F}_{\text{AUTH}}$, $\mathcal{F}_{\text{SMT}}^{l}$, and $\mathcal{F}_{\text{SYN}}$, respectively).

It has also proven its worth, both in its widespread use in proving security in real-world applications (see [Can+22] for a contemporaneous example), and its use in proving theorems that generalize over all UC-secure protocols in certain classes (see [CKL06], which proves the nonexistence of UC-secure protocols for certain two-party functionalities).

However, it is not without fault:

Complexity The UC paper has been revised numerous times; after being proposed in 2000; it was revised in 2001, 2005, 2013, twice in 2018, and yet again in 2020.

While over these revisions the framework has certainly matured, it has also accumulated a great deal of complexity – the paper now numbers over one hundred pages! While a great deal of this ink is devoted to discussion, a good deal of it is not, and the model ends up rather baroque by the time we reach the composition theorem:

> Let ρ, π, ϕ be PPT protocols and ξ be a PPT predicate such that ρ is (π, ϕ, ξ)-compliant and ϕ and π are both subroutine respecting and subroutine exposing, and π ξ-UC-emulates ϕ. Then protocol $\rho^{\phi \to \pi}$ UC-emulates protocol ρ.

Given the inherent difficulty of model-checking in this setting, it is not difficult to imagine that many security proofs using this definition miss some small subtlety of the model and end up incorrect.

[5] In this paper we consider *functionalities* and *ideal protocols* to essentially be interchangeable; this is not always strictly true formally, but is close enough for the purposes of this paper.

Trustworthiness These problems not only threaten the trustworthiness of proofs made of UC-security, but the security of UC itself. Its important theorem, the composition theorem, has not infrequently been challenged over the years:

- The 2005 version (*UC05*) has been challenged numerous times on orthogonal grounds, with issues with the definition of polynomial running time discussed in [HUM13] invalidating an important lemma used in the proof of composability and the theorem being flat out disproven using a counterexample in [HS15] due to issues with subroutine structure and the composition operation.
- The 2013 version (*UC13*) has been disproven with counterexample in[KTR20] due to issues with the formalization of the distinguishability experiment, leading it to gain powers over protocols used as subroutines it does not have over protocols analyzed on their own.
- Most recently, the 2020 version (*UC20*) has been claimed by [RKC22] to contain an oversight that again causes the composition theorem to fail with counterexample, due to the ability of the adversary to reveal its full code when communicating with party machines.

Other serious issues have also emerged; an issue with the definition of corruption and its application in the real and ideal worlds in UC05 made it almost impossible to (validly) prove the UC-security of any protocol, leading [HS15] to claim that "many (if not most) typical security claims in the literature are simply false."

Aside from the relevance of any individual flaw of the UC model, issues like these emerging again and again reveals a fundamental brittleness of the framework itself – slight variations in the description of UC can cause important results to become "simply false," and valid security proofs to become impossible. This, together with the complexity UC has acquired over the years makes it difficult to trust as an axiomatic definition of security.

3.2 Hofheinz and Shoup's GNUC

Recognizing many of the above shortfalls of UC, in 2013, Dennis Hofheinz and Victor Shoup published an alternative composability-focused framework entitled "**GNUC**": "*GNUC's Not Universal Composability*."[HS15] Recognizing that UC had "evolved a rather complex set of rules" that lead to "a number of undesirable effects," the aim was to rethink UC from the ground up, resulting in a framework "largely compatible with UC" but that deviates from UC "in several important aspects."

3.2.1 Overview

Libraries GNUC introduces the concept of *libraries*, which consist of finite mappings from program names to program code. In general, where UC considers arbitrary code, GNUC considers program names, with the consequence that only interactive machines (*IM*s, in GNUC's parlance) with specific code may be instantiated. It does however retain the ability for IMs to instanitate other IMs within these bounds.

Protocols and Subroutine Structure A protocol in GNUC is represented as a library of programs, each which specifies exactly which other programs may be called as subroutines. This is required to form a directed acyclic graph, so that each protocol IM has precisely one *caller* which it is a

subroutine of – no mutual subroutines here. The root or roots of this DAG are then taken to be the main machines of the protocol. GNUC also distinguishes between *regular* and *ideal machines*: whereas ideal machines are generally free to communicate, regular machines may only communicate with their subroutines, their caller, and the adversary. This forces communication between protocol parties to be modeled either as an ideal functionality or as an insecure channel mediated by the adversary.

Environment and Runtime The execution of a protocol under a given adversary and environment ends up being defined analogously to UC, with the environment being the main machine and at will being able to instantiate instances of the protocol and a single adversary, and at the end outputting a single bit. However, the notion of polynomial runtime ends up significantly different, with a concept of *flow* bounding the runtime of the entire system as a polynomial function of the total length of messages sent by the environment as well as the total length of messages sent by the environment to the adversary bounding the total length sent by adversaries to the protocol.

3.2.2 Discussion

In total, GNUC is clearly much more elegant and easier to understand. Its runtime properties are also quite nice, with the property that a protocol runs in polynomial time being composable (in UC and other systems, a poly-time protocol ϕ that uses a subroutine π may not remain poly-time when π is replaced with another poly-time subroutine π'). However, it is clearly much less flexible and much more limited than UC. There exist entire classes of functionalities it cannot fully realize, with the flow restrictions in particular preventing signature and encryption systems from being realized.

3.3 Küsters, Tuengerthal, and Rausch's IITM Model

Satisfied neither with UC nor with GNUC, in 2013 Küsters, Tuengerthal, and Rausch introduced their radically different IITM model.[KTR20]

3.3.1 Overview

Computation Model The IITM Model's computation model differs radically from both that of UC and GNUC. We call each interactive Turing machine M an *IITM*, and define a *system* of IITMs as a grouping of the form $M_1 \mid M_2 \mid \cdots \mid M_k \mid !M'_1 \mid !M'_2 \mid \cdots \mid !M'_j$. Within a system, IITMs prefixed by ! are permitted to be instantiated arbitrarily many times.[6] Each IITM lists its labeled communication input and output tapes, with each label occurring at most once as an input and once as an output in a system. These tapes form the basic communication mechanism, as opposed to the addressing system used by GNUC and UC.

When a message is sent to an IITM, it first runs some deterministic polytime computation to determine if it accepts the input. If the IITM it was sent to has a !, a message rejected by M' is passed to M'' and so on, until we reach an IITM that does not exist yet – in this case, a new M is instantiated, and if *this* one rejects it, control goes back to the main IITM.

This serves the dual purpose of acting as a program-specified addressing system and preventing attacks on protocol that aim to exhaust protocol machines by sending them too much garbage input – the latter reason is why IITMs are called IITMs: they're Inexhaustible Interactive Turing Machines.

[6] Note the similarities to GNUC's libraries.

Interfaces, Protocols, and Environments In a real world run of a protocol, an *environmental system* representing both the environment and the adversary and containing the main IITM connects to both the I/O and network tapes of a real *protocol system*, and the composed systems are run until a decision is reached. In an ideal run, the same experiment is run except replacing the real protocol with the ideal one, and with an (ideal) *adversarial system* mediating the communication between the environment's network tapes and the protocol's network tapes. A real protocol is said to **realize** an ideal protocol if there exists an ideal adversary such that the probability that the decision the environment reaches in each case is different is negligible, modulo some polynomial runtime restrictions on each system.

Composability The notion of composition of systems lends itself nicely to universal composition theorems, the first of which guarantees that security is preserved across the concurrent composition of a fixed number of protocol systems, and the second guaranteeing secure composition of an unbounded number of instances of a single protocol system, with the latter requiring some additional machinery to be defined in order to allow the coexistence of multiple sessions of a protocol system.

3.3.2 Discussion

Overall, IITM achieves a very elegant and expressive definition of security that remains under composition. Impressively, and unlike in the case of GNUC, this elegance does not come at the cost of expressiveness: [RKC22] shows that the UC model may be *embedded into* the IITM model, with the result being that "all protocols, security, and composability results from the UC model carry over to the IITM model."

What this elegance *may* come at the cost of, however, is usability and well-definition. Many such important aspects of modeling are left to protocol designers, with [Cam+19] providing a laundry list of aspects left unaddressed by the model: "party IDs, sessions, subroutine relationships, shared state, [and] (different forms of) corruption[,]" many of which lack even formal definitions in [KTR20]. Luckily, this same paper proposes a model known as *iUC* which elaborates on all of these from the foundation of the IITM model, which may alleviate these issues.

3.4 The Bottom-Up Strategy

These three definitions share a number of similarities – after all, they are all descendants of Canetti's UC Security framework. But on a broader level, they are similar because they are all defined in terms of Turing machines. As such, in some sense, they are all defined on the lowest possible level of abstraction. Any security researcher attempting to use these definitions on a higher level of abstraction will need to work their way up the ladder, defining their own more abstract security definitions and then proving that they are sound with regards to UC / GNUC / the IITM model.[7]

Veering towards more opinion-based and subjective territory, we claim that this bottom up strategy naturally leads to notions of security that take one hundred or more pages to describe, are highly sensitive to model-specific artifacts, and are not self-justifying.

[7] For example, see [CH06]

4 The Top-Down Strategy

As advocated in [MR11], the top-down strategy aims to abstract away most of the minute details for defining models of computation and communication, specific algorithms and their complexity and security for cryptographic schemes, as was needed in the bottom-up approach. In order to prove more general and stronger results without worrying about messy details, [MR11] proposed to start from an abstract system and prove more powerful and elegant results from that, leaving lower level details such as specific composability and efficiency to the specific system to define.

4.1 Abstract Cryptography

Abstract Cryptography [MR11], which first introduced abstraction into cryptography and defined what the top-down strategy entailed, paved the way for almost all current abstractions that we will see [Mau12], [MT13], [BK22] among other ones that have been done such as [LM18]. It achieved its goal of defining cryptography in a general fashion, with most definitions and statements being set and function theoretic, and proved a composability result and showed previous results, such as the one to due [Can20], were a special case of theirs. Due to its mathematical nature, we will have to introduce some machinery in a dry definition-theorem style to get to composability results and its use, which does imply we will have less drama but, as a trade-off, much more precision.

4.1.1 Reductions

As is commonly used with any complex system to analyze, modular or component systems provide a tractable way to build up to the whole with simpler components. This iterative paradigm of creating components from components with a constructor defining how to combine them is very commonly used in disciplines such as software design, web design, hardware, and machines, and so Maurer and Renner claim that should cryptography and security should be no different. This idea motivates the following definitions.

Definition 4.1. A **component set** is set Ω with (composition) operator $\|$. A **constructor set** is set Γ with (parallel composition) operator $|$, (serial composition) operator $\circ$, and (neutral) element id.

Definition 4.2. A **reduction** for component set Ω and constructor set Γ is subset $\Omega' \times \Gamma' \times \Omega'' \subseteq \Omega \times \Gamma \times \Omega$. If $(R, \alpha, S) \in \Omega' \times \Gamma' \times \Omega''$, then we write $R \xrightarrow{\alpha} S$ to mean S can be reduced from R via α. We write $R \to S$ if $\exists \alpha \in \Gamma$ such that $R \xrightarrow{\alpha} S$.

As an example, if given components R_1, R_2, we can reduce to a combined component of some sort S with a constructor α, then we would write

$$R_1 \parallel R_2 \xrightarrow{\alpha} S$$

and so $(\{R_1, R_2\}, \alpha, S)$ would be in the set of reductions. We can successively apply this process to create more and more complex systems as our initial goal was, of which the process we will call **stepwise refinement**. With these notions, our goal is still to establish when we can compose operations of reductions for Ω as a whole with Γ.

Remark 4.3. The reduction $\to$ for Ω and Γ forms a collection of relations on the elements of Ω index by elements of Γ where we can say that $R \sim_\alpha S$ if $R \xrightarrow{\alpha} S$.

Given this idea of relations on Ω, we can define what is means for a reduction $\rightarrow$ for Ω and Γ to be composable.

Definition 4.4. Reduction $\rightarrow$ for Ω and Γ is **serially composable** if $\forall R, S, T \in \Omega$ and $\alpha, \beta \in \Gamma$, we have

1. $R \xrightarrow{\alpha} S \wedge S \xrightarrow{\beta} T \Rightarrow R \xrightarrow{\alpha \circ \beta} T$
2. $R \xrightarrow{\mathsf{id}} S$

$\rightarrow$ is **context-insensitive** if

1. $R \xrightarrow{\alpha} S \Rightarrow R \parallel T \xrightarrow{\alpha | \mathsf{id}} S \parallel T \wedge T \parallel R \xrightarrow{\mathsf{id} | \alpha} T \parallel S$

If $\rightarrow$ is both serially composable and context-insensitive, then we simply say *to* is **generally composable**

One may notice the definition for serially composable is almost precisely that of a category by identifying components as objects and constructors as morphisms. Moreso, general composability may be viewed as a monoidal category. In fact, this is precisely what [BK22] studies by considering composition in he context of categories and abtracting resource theory. Content-insensitive is important for encoding that the reduction holds true independent of the environment and other potential mishaps that could be difficult to control. One result to see that this notion of general composability is indeed one we want is with a soundness result.

Theorem 4.5. *Reduction $\rightarrow$ is sound for step-wise refinement if and only if it is generally composable.*

where soundness holds if every local reduction in the tree from simple components to complex components is valid, then the reduction as a whole from the simplest components to the final complex component is valid. One can prove 4.5 via induction [MR11]. Notice the stark contrast in this definition with attempts at previous models where we don't worry about any minute details and take the most general approach with arbitrary sets Ω and Γ. A mathematically rigorous approach ensure rigorousness in argument and allows one to make precise statements.

4.1.2 Settings and Specifications

Consider a general setting where n parties each make a choice from which there may be an effect. Let A_i be the set of choices party i may make called the **ith choice domain**. This notion of choice with effect is essentially modeled as an n-ary function.

Definition 4.6 (n-choice setting)**.** An **n-choice setting** or **n-setting** is a map

$$A_1 \times A_2 \times \cdots \times A_n \rightarrow (\Omega, \delta)$$

where (Ω, δ) is a pseudo-metric called the **effect space**. δ induces an equivalence class on Ω given by $w \equiv w'$ if and only if $\delta(w, w') = 0$.

The pseudo-metric space will be useful later for developing approximations and working towards cryptographic algebras. For now, we will mainly use the equivalence class. Between n-choice settings, we can define a notion of isomorphisms called *complete factorizable relation (CFR)*. The

technical definition is a bit unruly, but it essentially says that for CFR ρ, given choice sets A_i, B_i, there is a relabeling given by ρ between $a_j \in A_i$ to $b_j \in B_j$ such that any n-choice settings i.e. maps $\Pi_{i=1}^n A_i \mapsto \Omega$ and $\Pi_{i=1}^n B_i \mapsto \Omega$ are isomorphic or give an equivalent result under the induced equivalence relation by δ. Oftentimes, we are only interested in a subset of settings, so we introduce the notion of specifications

Definition 4.7 (n-choice setting specification)**.** An **n-choice setting specification** or **n-specification** is a set of settings with an n-tuple of sets $(\widehat{A_1}, \widehat{A_2}, \cdots, \widehat{A_n}) \subseteq \Pi_{i=1}^n A_i$ where $\widehat{A_i}$ is a **guaranteed choice domain**.

One can imagine a specification as a restriction of the entire setting space since it essentially represents only the set of settings that we care about in a particular instance. An example that relates to computational security would be taking the space of all possible adversary actions as the settings and taking the specification of polynomial attacks.

Definition 4.8. For n-specifications $\mathcal{R}$ and $\mathcal{S}$ with guaranteed choice domains $\widehat{A_1}, \cdots, \widehat{A_n}$ and $\widehat{B_1}, \cdots, \widehat{B_n}$, let $\pi_i : \widehat{A_i} \to \widehat{B_i}$ and $\pi = (\pi_1, \cdots, \pi_n)$. $\mathcal{S}$ is a **π-abstraction** of $\mathcal{R}$, which we denote $R \sqsubseteq^\pi S$, if $\forall R \in \mathcal{R}, \exists S \in \mathcal{S}$ and CFR $\rho = (\rho_1, \cdots, \rho_n)$ such that

1. $R \cong_\rho S$
2. $\pi_i^{-1} \subseteq \rho_i \forall i$

Roughly, this definition says that elements of $\mathcal{R}$ are isomorphic to $\mathcal{S}$ and the inverse map $\pi_i^{-1} : \widehat{A_i} \to \widehat{B_i}$ is contained in the isomorphism. One should view the relation $R \sqsubseteq^\pi S$ to mean $\mathcal{S}$ is an abstraction of $\mathcal{R}$. We often say that $\mathcal{R}$ is the *concrete specification* and $\mathcal{S}$ is the *abstraction specification*. The power of this is that if one analyzes $\mathcal{S}$ to find a cryptographic scheme is secure, then $\mathcal{R}$ will similarly have the cryptographic scheme secure. This sounds a lot like a reduction, and, in fact, we can prove it is one.

Theorem 4.9. *$R \sqsubseteq^\pi S$ is a generally composable reduction.*

We can construct it with the component set as all specifications and constructor set as maps between $\Pi_{i=1}^n A_i$, n-tuples of choice domains. For the operators, composition of components can be given by an embedding $\|: \Omega \times \Omega$, from which we can apply $\|$ to the product $R \times S$ to map back down to Ω. Serial and parallel composition is given naturally by function composition $\circ$ and product $\times$ for constructors, and id is simple the identity function. Completing the proof can be done by checking the axioms.

4.1.3 Systems Theory[8]

To establish a cryptographic system with high generality, we move onto the discussion of the theory of systems. In full generality, a system is some abstract objection with interfaces that interact with the outside world. We consider 3 types of systems defined below.

Definition 4.10. A **resource system** (**resource**) R is a system with n interfaces labelled $I = \{1, 2, \cdots, n\}$.

[8]Not to be confused with *systems* in IITM

Definition 4.11. A **converter system** (**converter**) α is a system with 2 interfaces, where one is the inside interface and one the outside interface. It can be merged with a resource by connecting the inside interface to interface $i \in I$ of the resource to create a new resource where the outside interface is the new interface i of the resource. We write $\alpha^i R$ for the resulting resource.

Definition 4.12. A **distinguisher system** (**distinguisher**) D for a n-interface resource is a system with $n + 1$ interfaces, n of which are inside interfaces and 1 is outside. The n inside interfaces can connect to a resource and the outside interface returns a bit. We write DR for the resulting resource. Note, however, that DR is a binary random variable.

We can view these as abstract systems building blocks. Roughly, a resource simple holds resources, converters can convert between resources and exchange information, and distinguishers are used to distinguish between resources. If we let $\mathcal{D}$ be the class of all distinguishers, then we can define the advantage of $\mathcal{D}$ for resources R, S to be

$$\Delta^{\mathcal{D}}(R, S) = \sup_{D \in \mathcal{D}} \Delta^{D}(R, S)$$

where the advantage of D is Δ^D, a statistical distance between DR, DS. One can imagine fixing a set of $\mathcal{D}$ to fit one's uses to achieve adaptable security against any adversaries, such as statistical security as all security or computational security as PPT distinguishers. If $\mathcal{D}$ has 0 advantage for R, S, then we write $R \approx S$, which forms an equivalence relation on resources. Now, with systems, we can define our main object for describing cryptographic systems.

Definition 4.13. A **cryptographic algebra** $(\Phi, \Sigma, \approx)$ for interfaces I consists of set Φ of I-resources with composition $\|$ (which may be defined by a sort of merging), set Σ of converters, and an equivalence relation $\approx$ on Φ. Additionally, we want a map $\Sigma \times \Phi \times I \to \Phi$ defining the output resource when we attack a converter to an interface of a resource such that

1. Converters commute: $\alpha^i \beta^j R \approx \beta^j \alpha^i R$
2. Attaching no converter is defined as a *neutral* converter $\mathbf{1}$: $1^i R \approx R$
3. $R \approx S \Rightarrow \alpha^i R \approx \alpha^i S$
4. $R \approx S \Rightarrow R \parallel T \approx S \parallel T$ and $T \parallel R \approx T \parallel S$

Notice that in a cryptographic algebra, each resource system is a choice setting with choices of converters and effects of the resulting resource when applying the converter. With this idea and another of *filtered choices*, where essentially every choice setting is filtered to just the guaranteed choices, we can connect crytopgraphic algebras with choice settings to give:

Theorem 4.14. *For cryptographic algebra* $(\Phi, \Sigma, \approx)$ *and any* $i \in I$*, take converters* $\varphi_i, \psi_i, \pi_i, \sigma_i \in \Sigma$*,*

$$\forall P \subseteq I, \pi_P \varphi_P \approx \sigma_{\overline{P}} \psi_P S \Rightarrow R_\phi \sqsubseteq_\pi S_\psi$$

which is the main result. 4.14 gives us a way to construct abstractions and thus reductions. With this, we can view S_ψ as the "ideal world" and R_ϕ as the "real world" to get guaranteed proofs by the definition of π-abstractions. Note that the construction is local as well, since it only relies on simulations between R, S, which lends itself nicely to compositions.

4.1.4 Discussion

Certainly, we find much more precise notions and rigorous proofs here. Everything is well-defined with sets and with high generality. Essentially nothing is assumed about the system, so the results proven may be used on any cryptosystem, though this may have its own drawbacks in that it increases difficulty for using these results. Additionally, the full generality approach implies that we can't prove as strong claims, such as a form of composibility always holding.

4.2 Constructive Cryptography & Extensions

Constructive cryptography [Mau12] was developed in parallel with abstract cryptography [MR11], though builds on it greatly. Constructive crytpography, as a whole, takes a wholly *resource theoretic* view and focuses more on the ability to *construct* ideal resources from real resources. Some examples of this would be defining security as the construction of a system that satisfies desirable properties from resources that one possesses, such as defining a message authentication code (MAC) to be secure if it *constructs* an authenticated channel from an insecure channel and secret key. This differs greatly from the typical fluctuating definition from an adversarial view, which involves different possible powers of adversaries including computational power, as shown by the difference between statistical vs. computational security, and how much information they possess, such as chosen-plaintext attacks (CPA) or ciphertext-only attack (COA).

There additional extensions that have taken alternative routes such as a domain-theoretic route [MT13] and category-theoretic route [BK22]. [MT13] takes a slightly different approach of fixing a concrete computational model and modeling processes as monotonic functions with respect to time. From this idea, Micciancio and Tessaro can model systems, such as communication channels with posets ordered by prefix. [BK22] continues with a resource theoretic view but formalized in the language of symmetric monoidal categories.

5 Conclusion

A wise twitter user once said that "[c]ryptography is nightmare magic math that cares what kind of pen you use."[Swi17]. In this paper, we considered various types of foundational pens one could use and concluded, in our subjective view, that perhaps not considering the type of pen may be the most reasonable view.

References

[BP06] Bruno Blanchet and David Pointcheval. "Automated Security Proofs with Sequences of Games". en. In: *Advances in Cryptology - CRYPTO 2006*. Ed. by Cynthia Dwork. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2006, pp. 537–554. ISBN: 978-3-540-37433-6. DOI: 10.1007/11818175_32.

[BK22] Anne Broadbent and Martti Karvonen. "Categorical composable cryptography". en. In: *Foundations of Software Science and Computation Structures*. Ed. by Patricia Bouyer and Lutz Schröder. Vol. 13242. Series Title: Lecture Notes in Computer Science. Cham: Springer International Publishing, 2022, pp. 161–183. ISBN: 978-3-030-99252-1 978-3-030-99253-8. DOI: 10.1007/978-3-030-99253-8_9. URL: https://link.springer.com/10.1007/978-3-030-99253-8_9 (visited on 11/29/2022).

[Cam+19] Jan Camenisch et al. "iUC: Flexible Universal Composability Made Simple". en. In: *Advances in Cryptology – ASIACRYPT 2019*. Ed. by Steven D. Galbraith and Shiho Moriai. Lecture Notes in Computer Science. Cham: Springer International Publishing, 2019, pp. 191–221. ISBN: 978-3-030-34618-8. DOI: 10.1007/978-3-030-34618-8_7.

[Can20] Ran Canetti. "Universally Composable Security". In: *Journal of the ACM* 67.5 (Sept. 2020), 28:1–28:94. ISSN: 0004-5411. DOI: 10.1145/3402457. URL: https://doi.org/10.1145/3402457 (visited on 12/07/2022).

[CH06] Ran Canetti and Jonathan Herzog. "Universally Composable Symbolic Analysis of Mutual Authentication and Key-Exchange Protocols". en. In: *Theory of Cryptography*. Ed. by Shai Halevi and Tal Rabin. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2006, pp. 380–403. ISBN: 978-3-540-32732-5. DOI: 10.1007/11681878_20.

[CKL06] Ran Canetti, Eyal Kushilevitz, and Yehuda Lindell. "On the Limitations of Universally Composable Two-Party Computation Without Set-Up Assumptions". en. In: *Journal of Cryptology* 19.2 (Apr. 2006), pp. 135–167. ISSN: 1432-1378. DOI: 10.1007/s00145-005-0419-9. URL: https://doi.org/10.1007/s00145-005-0419-9 (visited on 12/06/2022).

[Can+22] Ran Canetti et al. "Universally Composable End-to-End Secure Messaging". en. In: *Advances in Cryptology – CRYPTO 2022*. Ed. by Yevgeniy Dodis and Thomas Shrimpton. Lecture Notes in Computer Science. Cham: Springer Nature Switzerland, 2022, pp. 3–33. ISBN: 978-3-031-15979-4. DOI: 10.1007/978-3-031-15979-4_1.

[HS15] Dennis Hofheinz and Victor Shoup. "GNUC: A New Universal Composability Framework". en. In: *Journal of Cryptology* 28.3 (July 2015), pp. 423–508. ISSN: 1432-1378. DOI: 10.1007/s00145-013-9160-y. URL: https://doi.org/10.1007/s00145-013-9160-y (visited on 12/07/2022).

[HUM13] Dennis Hofheinz, Dominique Unruh, and Jörn Müller-Quade. "Polynomial Runtime and Composability". en. In: *Journal of Cryptology* 26.3 (July 2013), pp. 375–441. ISSN: 1432-1378. DOI: 10.1007/s00145-012-9127-4. URL: https://doi.org/10.1007/s00145-012-9127-4 (visited on 12/20/2022).

[KTR20] Ralf Küsters, Max Tuengerthal, and Daniel Rausch. "The IITM Model: A Simple and Expressive Model for Universal Composability". en. In: *Journal of Cryptology* 33.4 (Oct. 2020), pp. 1461–1584. ISSN: 1432-1378. DOI: 10.1007/s00145-020-09352-1. URL: https://doi.org/10.1007/s00145-020-09352-1 (visited on 12/07/2022).

[LM18] Baiyu Li and Daniele Micciancio. "Equational Security Proofs of Oblivious Transfer Protocols". en. In: *Public-Key Cryptography – PKC 2018*. Ed. by Michel Abdalla and Ricardo Dahab. Lecture Notes in Computer Science. Cham: Springer International Publishing, 2018, pp. 527–553. ISBN: 978-3-319-76578-5. DOI: 10.1007/978-3-319-76578-5_18.

[Lin17] Yehuda Lindell. "How to Simulate It – A Tutorial on the Simulation Proof Technique". en. In: *Tutorials on the Foundations of Cryptography: Dedicated to Oded Goldreich*. Ed. by Yehuda Lindell. Information Security and Cryptography. Cham: Springer International Publishing, 2017, pp. 277–346. ISBN: 978-3-319-57048-8. DOI: 10.1007/978-3-319-57048-8_6. URL: https://doi.org/10.1007/978-3-319-57048-8_6 (visited on 11/28/2022).

[Mau12] Ueli Maurer. "Constructive Cryptography – A New Paradigm for Security Definitions and Proofs". en. In: *Theory of Security and Applications*. Ed. by Sebastian Mödersheim and Catuscia Palamidessi. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2012, pp. 33–56. ISBN: 978-3-642-27375-9. DOI: 10.1007/978-3-642-27375-9_3.

[MR11] Ueli Maurer and Renato Renner. "Abstract Cryptography". In: *International Conference on Supercomputing*. 2011.

[MT13] Daniele Micciancio and Stefano Tessaro. "An equational approach to secure multi-party computation". In: *Proceedings of the 4th conference on Innovations in Theoretical Computer Science*. ITCS '13. New York, NY, USA: Association for Computing Machinery, Jan. 2013, pp. 355–372. ISBN: 978-1-4503-1859-4. DOI: 10.1145/2422436.2422478. URL: https://doi.org/10.1145/2422436.2422478 (visited on 11/27/2022).

[RKC22] Daniel Rausch, Ralf Kuesters, and Céline Chevalier. *Embedding the UC Model into the IITM Model*. Report Number: 224. 2022. URL: https://eprint.iacr.org/2022/224 (visited on 12/20/2022).

[Swi17] SwiftOnSecurity [@SwiftOnSecurity]. *Tried explaining danger of homebrew crypto to a journalist in DM "Cryptography is nightmare magic math that cares what kind of pen you use."* en. Tweet. Feb. 2017. URL: https://twitter.com/SwiftOnSecurity/status/832055497251487744 (visited on 12/21/2022).